Singly Linked List – Search Element In Sorted List Recursive – Space Complexity and Algorithm.

Program:

```
Node *searchInUnsortedList(Node **head, int key)
{
    Node *current = *head;
    while (current != nullptr && current->data != key)
    {
        current = current->next;
    }
    return current;
}
……
 case 13:
        {
            int key;
            cout << "Enter the element to search: ";
            cin >> key;
            if (searchInUnsortedList(&head, key) != nullptr)
            {
                Node *cur = searchInUnsortedList(&head, key);
                cout << "Element is:" << cur->data << endl;
                cout << "Element found." << endl;

            }
            else
            {
                cout << "Element not found." << endl;
            }
            break;
        }
```

Space Complexity Analysis

1. *Input Space Complexity*:

A. Parameter – Only Definition:

The input space complexity refers to the space required to store the input parameters of the function. In this case, the function takes two parameters:

- ***Node ** head***: ***A pointer to the head of the linked list***.
- ***int key***: ***An integer representing the key to search for***.

Both of these parameters require a constant amount of space, i.e., O(1).

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer-to-a-pointer) and `int key` (an integer), both of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

• Node **head → one pointer (machine word)

• int key → one integer (machine word) ⇒ Θ(1) = O(1)

B. Full – Data Definition:

This more common definition includes the entire data structure. If the linked list has **`n` nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

• Same two words as above (Θ(1)), plus

• Every node that can be reached from *head when the call starts. If the list contains n nodes, they occupy Θ(n) bytes. ⇒ Θ(n) = O(n)

2. *Auxiliary Space Complexity*:

Auxiliary space complexity refers to the extra space used by the algorithm, excluding the input space. In this function, the only additional space used is for the pointer Node * current, which is used to traverse the linked list.

Since current is a single pointer, it also requires a constant amount of space, i.e., $O(1)$.

• Local variable: – Node *current (plus a few compiler temporaries) → constant size

• No dynamic allocations, no recursion.

⇒ $\Theta(1) = O(1)$

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this iterative search function:

1. **`Node *current`**: The function declares only **one** extra variable, a pointer named `current`, to traverse the list. This pointer occupies a constant amount of memory.
2. **Iteration**: The `while` loop reuses this single `current` pointer to move through the list. No new memory is allocated during the search.

Since the function only needs one extra pointer to do its job, regardless of the list's length, the amount of extra memory used is constant.

Therefore, the auxiliary space complexity is **$\Theta(1)$**.

Algorithm :

$\boldsymbol{SrchInUnsrtdList(START, KEY)}$

$\boldsymbol{Set\ CURRPTR := START\ [Sets\ CURRPTR\ pointer\ to\ \ NODE\ pointed\ by\ HEAD\ or\ START]}$

$\boldsymbol{Repeat\ while\ CURRPTR\ \neq NULL\ AND\ INFO[CURRPTR\ ]\ \neq KEY, then:}$

$\boldsymbol{Set\ CURRPTR\ := LINK[CURRPTR]\ [It\ sets\ CURRPTR\ pointer\ to\ the\ address\ of\ NODE\ where\ KEY\ is\ located]}$

$\boldsymbol{[Otherwise\ , if\ KEY\ is\ not\ found\ then, CURRPTR = NULL, LOOP\ ends]}$

$\boldsymbol{[End\ of\ Loop]}$

$\boldsymbol{return\ CURRPTR}$